

LAKSHMI NARAIN COLLEGE OF TECHNOLOGY

Department of Computer Science & Engineering

ASSIGNMENT REPORT

Submitted By

VARUN JUNEJA

College	LNCT
Enrollment Number	0103CS231449
Superset ID	7809995
Email	varunjuneja1905@gmail.com

Academic Session 2025 – 2026

EXPERIMENT NO. 1

TITLE: Introduction to HQL and JPQL

Aim

To study and implement Hibernate Query Language (HQL) and Java Persistence Query Language (JPQL) using Spring Data JPA.

Objectives

1. To understand HQL and JPQL concepts.
 2. To compare HQL with SQL.
 3. To execute entity-based queries.
 4. To use `@Query` annotation in Spring Data JPA.
 5. To retrieve data using object-oriented queries.
-

Prerequisites

- Java 17
 - Spring Boot 3.x
 - Maven
 - MySQL Server
 - Spring Data JPA
 - IntelliJ IDEA/Eclipse
-

Theory

Hibernate Query Language (HQL) is an object-oriented query language provided by Hibernate framework. HQL resembles SQL syntax but works with persistent Java objects and their properties instead of database tables and columns.

Java Persistence Query Language (JPQL) is the standard query language defined by JPA specification. JPQL is vendor independent and portable across various JPA implementations.

Comparison between SQL, HQL and JPQL

Feature	SQL	HQL	JPQL
Works On	Tables	Entities	Entities
Standard	ANSI	Hibernate	JPA
Database Dependent	Yes	No	No

Feature	SQL	HQL	JPQL
INSERT Support	Yes	Yes	No
Portability	Low	Medium	High

Advantages

- Reduces database dependency.
- Works directly with Java objects.
- Supports inheritance and relationships.
- Improves code readability.

Architecture / Working Diagram

[Insert HQL Architecture Diagram Here]

Scenario Description

An employee management application stores employee information in a relational database. HQL/JPQL queries are used to retrieve employee details using entity classes.

Project Structure



Files Created

- Employee.java
- EmployeeRepository.java
- EmployeeService.java
- OrmLearnApplication.java
- application.properties
- pom.xml

Implementation

Entity Class

```
@Entity
@Table(name = "employee")
public class Employee {

    @Id
    private int id;

    private String name;

    private double salary;

    public Employee() {}

    public Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }

    @Override
    public String toString() {
        return "Employee{" +
            "id=" + id +
            ", name='" + name + '\'' +

```

```
        ", salary=" + salary +  
        '}'';  
    }  
}
```

Repository Layer

```
@Repository  
public interface EmployeeRepository  
    extends JpaRepository<Employee, Integer> {  
  
    @Query("SELECT e FROM Employee e")  
    List<Employee> getEmployees();  
}
```

Service Layer

```
@Service  
public class EmployeeService {  
  
    @Autowired  
    private EmployeeRepository repository;  
  
    public List<Employee> getEmployees() {  
        return repository.getEmployees();  
    }  
}
```

Main Application Class

```
@SpringBootApplication  
public class OrmLearnApplication {  
  
    @Autowired  
    private EmployeeService employeeService;  
  
    @PostConstruct  
    public void test() {  
  
        List<Employee> employees =  
            employeeService.getEmployees();  
  
        employees.forEach(System.out::println);  
    }  
  
    public static void main(String[] args) {
```

```
        SpringApplication.run(  
            OrmLearnApplication.class, args);  
    }  
}
```

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn  
spring.datasource.username=root  
spring.datasource.password=root  
  
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.properties.hibernate.format_sql=true
```

pom.xml Dependencies

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-data-jpa</artifactId>  
</dependency>  
  
<dependency>  
    <groupId>com.mysql</groupId>  
    <artifactId>mysql-connector-j</artifactId>  
</dependency>
```

Code Explanation

The repository layer contains an HQL query using the `@Query` annotation. Spring Data JPA converts the HQL query into SQL automatically and executes it.

Query Explanation

```
SELECT e FROM Employee e
```

- `Employee` refers to entity class.
 - `e` is alias.
 - Entire `Employee` objects are returned.
 - No table name is used.
-

Execution Steps

1. Create Spring Boot project.
 2. Add Spring Data JPA dependency.
 3. Configure MySQL database.
 4. Create Employee entity.
 5. Create repository interface.
 6. Define HQL query.
 7. Execute application.
 8. Verify console output.
-

Sample Input

ID	Name	Salary
1	John	60000
2	Sam	50000

Sample Output

```
Employee{id=1, name='John', salary=60000.0}  
Employee{id=2, name='Sam', salary=50000.0}
```

Screenshots

[Insert Screenshot 1 Here]

[Insert Screenshot 2 Here]

[Insert Screenshot 3 Here]

Advantages

1. Database independent.
 2. Object-oriented querying.
 3. Improves maintainability.
 4. Reduces SQL dependency.
-

Limitations

1. Learning curve exists.
 2. Complex joins are difficult.
 3. Hibernate specific features reduce portability.
-

Viva Questions and Answers

Q1. What is HQL?

HQL is Hibernate Query Language used to query entities.

Q2. What is JPQL?

JPQL is Java Persistence Query Language defined by JPA.

Q3. Is JPQL a subset of HQL?

Yes.

Q4. Does HQL use table names?

No, it uses entity names.

Q5. Which annotation is used to define HQL?

@Query

Result

HQL and JPQL queries were successfully implemented and employee records were retrieved using Spring Data JPA.

EXPERIMENT NO. 2

TITLE: Get All Permanent Employees using HQL

Aim

To retrieve all permanent employees using HQL along with department and skills.

Objectives

1. To write HQL queries using @Query.
 2. To understand fetch joins.
 3. To avoid N+1 query problem.
 4. To retrieve associated entities.
 5. To optimize HQL queries.
-

Prerequisites

- Spring Boot

- Spring Data JPA
- Hibernate
- MySQL
- Java 17

Theory

HQL supports object-oriented joins similar to SQL joins. When entities are associated through relationships, Hibernate may execute multiple SQL queries to fetch related data.

The **fetch** keyword forces Hibernate to load related entities immediately.

Without Fetch

- Multiple SQL queries execute.
- Causes N+1 problem.
- Performance degrades.

With Fetch

- Single optimized query executes.
- Related entities load together.
- Better performance.

Architecture / Working Diagram

[Insert Employee-Department-Skill Relationship Diagram Here]

Scenario Description

An organization maintains employee details along with department and skill information. The application should retrieve only permanent employees and their associated information.

Project Structure

```
entity
├── Employee.java
├── Department.java
└── Skill.java

repository
└── EmployeeRepository.java

service
└── EmployeeService.java
```

Files Created

- Employee.java
 - Department.java
 - Skill.java
 - EmployeeRepository.java
 - EmployeeService.java
 - OrmLearnApplication.java
-

Database Schema

Employee

Column
em_id
em_name
em_salary
em_permanent
em_dp_id

Department

Column
dp_id
dp_name

Skill

Column
sk_id
sk_name

Entity Relationship Explanation

- One Department can contain many Employees.
 - One Employee can possess multiple Skills.
 - Employee and Skill have Many-to-Many relationship.
-

Implementation

Department Entity

```
@Entity
public class Department {

    @Id
    private int id;

    private String name;

    @OneToMany(mappedBy = "department",
        fetch = FetchType.LAZY)
    private List<Employee> employeeList;
}
```

Skill Entity

```
@Entity
public class Skill {

    @Id
    private int id;

    private String name;
}
```

Employee Entity

```
@Entity
public class Employee {

    @Id
    private int id;

    private String name;

    private double salary;

    private boolean permanent;

    @ManyToOne
    private Department department;

    @ManyToMany
    @JoinTable(
        name = "employee_skill",
        joinColumns =
            @JoinColumn(name = "es_em_id"),
```

```
        inverseJoinColumns =
            @JoinColumn(name = "es_sk_id"))
    private List<Skill> skillList;
}
```

Repository Layer

```
@Repository
public interface EmployeeRepository
    extends JpaRepository<Employee,Integer> {

    @Query("""
        SELECT e
        FROM Employee e
        LEFT JOIN FETCH e.department
        LEFT JOIN FETCH e.skillList
        WHERE e.permanent = true
        """)
    List<Employee> getAllPermanentEmployees();
}
```

Service Layer

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repository;

    public List<Employee>
    getAllPermanentEmployees() {

        return repository
            .getAllPermanentEmployees();
    }
}
```

Test/Application Layer

```
@PostConstruct
public void testPermanentEmployees() {

    List<Employee> employees =
        employeeService
            .getAllPermanentEmployees();
}
```

```
employees.forEach(e -> {  
    System.out.println(e);  
    System.out.println(e.getSkillList());  
});  
}
```

Code Explanation

The query uses **LEFT JOIN FETCH** to retrieve department and skills along with employee details in one query.

Query Explanation

```
SELECT e  
FROM Employee e  
LEFT JOIN FETCH e.department  
LEFT JOIN FETCH e.skillList  
WHERE e.permanent = true
```

Explanation

- **LEFT JOIN** joins associated entities.
- **FETCH** loads associated objects.
- **WHERE** filters permanent employees.

Why Fetch is Used?

Fetch ensures associated entities are loaded immediately and prevents additional SQL execution.

Optimization

Using fetch join eliminates multiple select statements and avoids N+1 problem.

Execution Steps

1. Create entities.
 2. Configure relationships.
 3. Create repository.
 4. Add HQL query.
 5. Create service method.
 6. Execute application.
 7. Observe SQL logs.
 8. Verify output.
-

Sample Output

```
Employee{id=1, name='John'}  
  
Department : IT  
  
Skills : [Java, Spring, SQL]
```

Screenshots

[Insert Screenshot 1 Here]

[Insert Screenshot 2 Here]

[Insert Screenshot 3 Here]

Advantages

1. Single query execution.
 2. Better performance.
 3. Avoids lazy initialization issues.
 4. Eliminates N+1 problem.
-

Limitations

1. Large joins may impact memory.
 2. Complex queries reduce readability.
 3. Over-fetching can occur.
-

Viva Questions and Answers

Q1. What is Fetch Join?

Fetch Join loads associated entities immediately.

Q2. Why is Fetch Join used?

To improve performance.

Q3. What is N+1 problem?

Multiple unwanted SQL queries executed for associated entities.

Q4. Which annotation defines HQL?

`@Query`

Q5. Which relationship exists between Employee and Skill?

Many-to-Many.

Result

Permanent employees along with department and skills were successfully retrieved using optimized HQL fetch joins.

EXPERIMENT NO. 3

TITLE: Fetch Quiz Attempt Details using HQL

Aim

To fetch complete quiz attempt details using HQL with fetch joins.

Objectives

1. To implement complex HQL queries.
 2. To understand nested entity relationships.
 3. To use multiple fetch joins.
 4. To retrieve complete object graphs efficiently.
 5. To avoid LazyInitializationException.
-

Prerequisites

- Java 17
 - Spring Boot 3.x
 - Spring Data JPA
 - Hibernate ORM
 - MySQL Database
 - Maven
-

Theory

In enterprise applications, data is often distributed across multiple related tables. Retrieving complete information requires joining several entities.

Hibernate provides **JOIN FETCH** to eagerly load related entities in a single query.

Without fetch joins:

- Multiple SQL statements are executed.
- Performance decreases.
- LazyInitializationException may occur.

With fetch joins:

- Single optimized query executes.
- Related collections are populated automatically.
- Better performance is achieved.

Quiz applications generally consist of:

- User
- Attempt
- AttemptQuestion
- Question
- AttemptOption
- Option

All these entities are connected using One-To-Many and Many-To-One relationships.

Architecture / Working Diagram

[Insert Quiz Entity Relationship Diagram Here]

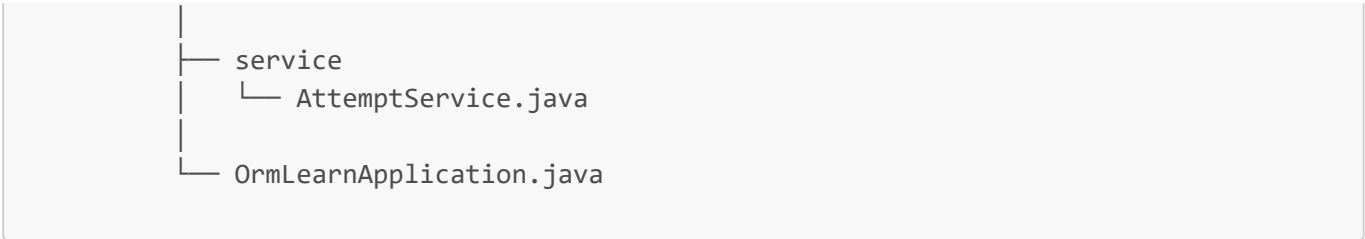
Scenario Description

An administrator needs to view complete quiz attempt details submitted by a user. The report should display:

- Username
 - Attempt Date
 - Questions attempted
 - Available options
 - Correct answer
 - Marks
 - Selected answer
-

Project Structure

```
src
├── main
│   └── java
│       ├── entity
│       │   ├── User.java
│       │   ├── Attempt.java
│       │   ├── AttemptQuestion.java
│       │   ├── Question.java
│       │   ├── AttemptOption.java
│       │   └── Option.java
│       ├── repository
│       │   └── AttemptRepository.java
```



Files Created

- User.java
- Attempt.java
- AttemptQuestion.java
- Question.java
- Option.java
- AttemptOption.java
- AttemptRepository.java
- AttemptService.java
- OrmLearnApplication.java

Database Schema

USER

Column
us_id
us_name
us_email

ATTEMPT

Column
at_id
at_date
at_us_id
at_score

QUESTION

Column
qt_id
qt_text

OPTIONS

Column

op_id

op_qt_id

op_text

op_score

ATTEMPT_QUESTION

Column

aq_id

aq_at_id

aq_qt_id

ATTEMPT_OPTION

Column

ao_id

ao_aq_id

ao_op_id

ao_selected

Entity Relationship Explanation

Parent Entity	Child Entity	Relationship
User	Attempt	One-To-Many
Attempt	AttemptQuestion	One-To-Many
Question	Option	One-To-Many
AttemptQuestion	AttemptOption	One-To-Many

Implementation

User Entity

```
@Entity
@Table(name = "user")
public class User {
```

```
@Id
@Column(name = "us_id")
private int id;

@Column(name = "us_name")
private String name;

@Column(name = "us_email")
private String email;

@OneToMany(mappedBy = "user")
private List<Attempt> attempts;
}
```

Attempt Entity

```
@Entity
@Table(name = "attempt")
public class Attempt {

    @Id
    @Column(name = "at_id")
    private int id;

    @Column(name = "at_date")
    private LocalDate attemptDate;

    @Column(name = "at_score")
    private double score;

    @ManyToOne
    @JoinColumn(name = "at_us_id")
    private User user;

    @OneToMany(mappedBy = "attempt")
    private List<AttemptQuestion> attemptQuestions;
}
```

Question Entity

```
@Entity
@Table(name = "question")
public class Question {

    @Id
    @Column(name = "qt_id")
```

```
private int id;

@Column(name = "qt_text")
private String text;

@OneToMany(mappedBy = "question")
private List<Option> options;
}
```

Option Entity

```
@Entity
@Table(name = "options")
public class Option {

    @Id
    @Column(name = "op_id")
    private int id;

    @Column(name = "op_text")
    private String text;

    @Column(name = "op_score")
    private double score;

    @ManyToOne
    @JoinColumn(name = "op_qt_id")
    private Question question;
}
```

AttemptQuestion Entity

```
@Entity
@Table(name = "attempt_question")
public class AttemptQuestion {

    @Id
    @Column(name = "aq_id")
    private int id;

    @ManyToOne
    @JoinColumn(name = "aq_at_id")
    private Attempt attempt;

    @ManyToOne
    @JoinColumn(name = "aq_qt_id")
    private Question question;
}
```

```
@OneToMany(mappedBy = "attemptQuestion")
private List<AttemptOption> attemptOptions;
}
```

AttemptOption Entity

```
@Entity
@Table(name = "attempt_option")
public class AttemptOption {

    @Id
    @Column(name = "ao_id")
    private int id;

    @ManyToOne
    @JoinColumn(name = "ao_aq_id")
    private AttemptQuestion attemptQuestion;

    @ManyToOne
    @JoinColumn(name = "ao_op_id")
    private Option option;

    @Column(name = "ao_selected")
    private boolean selected;
}
```

Repository Layer

```
@Repository
public interface AttemptRepository
    extends JpaRepository<Attempt,Integer> {

    @Query("""
        SELECT DISTINCT a
        FROM Attempt a
        JOIN FETCH a.user u
        JOIN FETCH a.attemptQuestions aq
        JOIN FETCH aq.question q
        JOIN FETCH q.options o
        LEFT JOIN FETCH aq.attemptOptions ao
        WHERE u.id = :userId
        AND a.id = :attemptId
        """)
    Attempt getAttempt(
        @Param("userId") int userId,
```

```
        @Param("attemptId") int attemptId);  
    }
```

Service Layer

```
@Service  
public class AttemptService {  
  
    @Autowired  
    private AttemptRepository repository;  
  
    public Attempt getAttempt(int userId,  
                             int attemptId) {  
  
        return repository.getAttempt(  
            userId,  
            attemptId);  
    }  
}
```

Controller/Test Layer

```
@PostConstruct  
public void testAttempt() {  
  
    Attempt attempt =  
        attemptService.getAttempt(1,1);  
  
    System.out.println(  
        "User : " +  
        attempt.getUser().getName());  
  
    for (AttemptQuestion aq :  
        attempt.getAttemptQuestions()) {  
  
        System.out.println(  
            aq.getQuestion().getText());  
  
        for (Option option :  
            aq.getQuestion().getOptions()) {  
  
            System.out.println(  
                option.getText() + " "  
                + option.getScore());  
        }  
    }  
}
```

```
}  
}
```

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn  
spring.datasource.username=root  
spring.datasource.password=root  
  
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.properties.hibernate.format_sql=true
```

pom.xml Dependencies

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>  
    spring-boot-starter-data-jpa  
  </artifactId>  
</dependency>  
  
<dependency>  
  <groupId>com.mysql</groupId>  
  <artifactId>mysql-connector-j</artifactId>  
</dependency>
```

Code Explanation

The repository method joins all associated entities using **JOIN FETCH**. This ensures that all quiz-related data is loaded in a single query.

The **DISTINCT** keyword prevents duplicate Attempt objects.

HQL Query Explanation

```
SELECT DISTINCT a  
FROM Attempt a  
JOIN FETCH a.user u  
JOIN FETCH a.attemptQuestions aq  
JOIN FETCH aq.question q  
JOIN FETCH q.options o
```

```
LEFT JOIN FETCH aq.attemptOptions ao
WHERE u.id=:userId
AND a.id=:attemptId
```

Explanation

Clause	Purpose
JOIN FETCH a.user	Loads user details
JOIN FETCH attemptQuestions	Loads all questions
JOIN FETCH question	Loads question details
JOIN FETCH options	Loads options
LEFT JOIN FETCH	Loads selected options
WHERE	Filters user and attempt

Why Fetch is Used?

Fetch keyword ensures that associated collections are populated immediately.

Without fetch:

- Additional SQL queries execute.
- `LazyInitializationException` occurs.

Optimization

The entire object graph is loaded using a single query, significantly improving application performance.

Execution Steps

1. Create database schema.
2. Create entity classes.
3. Define relationships.
4. Create repository interface.
5. Write HQL with fetch joins.
6. Implement service layer.
7. Execute application.
8. Verify quiz details.

Sample Input

User ID : 1
Attempt ID : 1

Sample Output

User : John

What is the extension of HTML file?

- | | | |
|----------|-----|-------|
| 1) .xhtm | 0.0 | false |
| 2) .ht | 0.0 | false |
| 3) .html | 1.0 | true |
| 4) .htm | 0.0 | false |

What is maximum heading level?

- | | | |
|------|-----|-------|
| 1) 5 | 0.0 | false |
| 2) 3 | 0.0 | false |
| 3) 4 | 0.0 | false |
| 4) 6 | 1.0 | true |

Screenshots

[Insert Screenshot 1 Here]

[Insert Screenshot 2 Here]

[Insert Screenshot 3 Here]

Advantages

1. Retrieves complete object graph.
 2. Improves performance.
 3. Eliminates LazyInitializationException.
 4. Reduces database hits.
-

Limitations

1. Query becomes complex.
 2. Large joins consume memory.
 3. Difficult to maintain for huge schemas.
-

Viva Questions and Answers

Q1. What is LazyInitializationException?

Exception thrown when lazy collection is accessed outside session.

Q2. Why is DISTINCT used?

To remove duplicate root entities.

Q3. What is Fetch Join?

Fetch Join retrieves associated entities immediately.

Q4. What annotation defines HQL?

`@Query`

Q5. Which relationship exists between Attempt and AttemptQuestion?

One-To-Many.

Result

Quiz attempt details along with user information, questions, options and selected answers were successfully retrieved using optimized HQL fetch joins.

EXPERIMENT NO. 4

TITLE: Get Average Salary using HQL

Aim

To compute the average salary of employees belonging to a particular department using HQL aggregate functions.

Objectives

1. To understand aggregate functions in HQL.
 2. To implement AVG() function using HQL.
 3. To use named parameters in HQL.
 4. To bind parameters using `@Param`.
 5. To calculate department-wise average salary.
-

Prerequisites

- Java 17
- Spring Boot 3.x

- Spring Data JPA
 - Hibernate ORM
 - MySQL Database
 - Maven
-

Theory

Hibernate Query Language supports aggregate functions similar to SQL.

Aggregate functions perform calculations on a set of values and return a single result.

Common aggregate functions:

- AVG()
- COUNT()
- MAX()
- MIN()
- SUM()

The **AVG()** function computes the arithmetic mean of numeric values.

HQL aggregate functions operate on entity attributes instead of database columns.

Named parameters improve readability and maintainability.

Architecture / Working Diagram

[Insert Aggregate Query Architecture Diagram Here]

Scenario Description

An organization wants to determine the average salary of employees working in a specific department. The administrator provides department id and the system computes the average salary.

Project Structure

```
src
├── main
│   └── java
│       ├── entity
│       │   ├── Employee.java
│       │   └── Department.java
│       ├── repository
│       │   └── EmployeeRepository.java
│       └── service
```

```
└─ EmployeeService.java
└─ OrmLearnApplication.java
```

Files Created

- Employee.java
- Department.java
- EmployeeRepository.java
- EmployeeService.java
- OrmLearnApplication.java
- application.properties

Database Schema

Employee Table

Column Name
em_id
em_name
em_salary
em_permanent
em_dp_id

Department Table

Column Name
dp_id
dp_name

Entity Relationship Explanation

- A Department can contain multiple Employees.
- Employee and Department are associated using Many-To-One relationship.

Implementation

Department Entity

```
@Entity
@Table(name = "department")
public class Department {

    @Id
    @Column(name = "dp_id")
    private int id;

    @Column(name = "dp_name")
    private String name;

    @OneToMany(mappedBy = "department")
    private List<Employee> employeeList;

    // Getters and Setters
}
```

Employee Entity

```
@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @Column(name = "em_id")
    private int id;

    @Column(name = "em_name")
    private String name;

    @Column(name = "em_salary")
    private double salary;

    @ManyToOne
    @JoinColumn(name = "em_dp_id")
    private Department department;

    // Getters and Setters
}
```

Repository Layer

```
@Repository
public interface EmployeeRepository
    extends JpaRepository<Employee, Integer> {
```

```
@Query("""
    SELECT AVG(e.salary)
    FROM Employee e
    WHERE e.department.id = :id
    """)
double getAverageSalary(
    @Param("id") int id);
}
```

Service Layer

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repository;

    public double getAverageSalary(int id) {

        return repository.getAverageSalary(id);
    }
}
```

Controller/Test Layer

```
@PostConstruct
public void testAverageSalary() {

    double avgSalary =
        employeeService.getAverageSalary(2);

    System.out.println(
        "Average Salary : " + avgSalary);
}
```

Main Application Class

```
@SpringBootApplication
public class OrmLearnApplication {

    public static void main(String[] args) {

        SpringApplication.run(
```

```
        OrmLearnApplication.class,  
        args);  
    }  
}
```

application.properties

```
spring.datasource.url=  
jdbc:mysql://localhost:3306/ormlearn  
  
spring.datasource.username=root  
spring.datasource.password=root  
  
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=update  
  
spring.jpa.properties.hibernate.format_sql=true
```

pom.xml Dependencies

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>  
    spring-boot-starter-data-jpa  
  </artifactId>  
</dependency>  
  
<dependency>  
  <groupId>com.mysql</groupId>  
  <artifactId>mysql-connector-j</artifactId>  
</dependency>
```

Code Explanation

The repository method uses the `AVG()` aggregate function to calculate the average salary.

The query filters employees based on department id.

The `@Param` annotation binds method parameters to query parameters.

HQL Query Explanation

```
SELECT AVG(e.salary)
FROM Employee e
WHERE e.department.id = :id
```

Explanation

Clause	Description
AVG()	Calculates average salary
e.salary	Employee salary attribute
WHERE	Filters employees
:id	Named parameter

Parameter Binding

```
@Param("id") int id
```

The `@Param` annotation maps the input parameter to the named parameter in HQL.

Aggregate Functions Supported in HQL

Function	Purpose
AVG()	Average value
SUM()	Total value
COUNT()	Number of rows
MAX()	Highest value
MIN()	Lowest value

Execution Steps

1. Create Department entity.
2. Create Employee entity.
3. Configure entity relationship.
4. Create repository interface.
5. Define aggregate HQL query.
6. Create service method.
7. Execute application.
8. Verify average salary.

Sample Input

```
Department Id = 2
```

Sample Output

```
Average Salary : 65000.0
```

Screenshots

[Insert Screenshot 1 Here]

[Insert Screenshot 2 Here]

[Insert Screenshot 3 Here]

Advantages

1. Performs efficient calculations.
 2. Reduces application-side processing.
 3. Improves performance.
 4. Easy to implement.
-

Limitations

1. Limited to aggregate operations.
 2. Complex calculations may require native SQL.
 3. Large datasets may impact performance.
-

Viva Questions and Answers

Q1. What is an aggregate function?

An aggregate function performs calculation on multiple values and returns a single value.

Q2. Which aggregate functions are supported in HQL?

AVG(), COUNT(), SUM(), MIN() and MAX().

Q3. Why is `@Param` used?

It binds method parameters with HQL parameters.

Q4. What does AVG() return?

It returns the average value.

Q5. Does HQL support aggregate functions?

Yes.

Result

The average salary of employees belonging to a particular department was successfully computed using HQL aggregate functions.

EXPERIMENT NO. 5

TITLE: Get All Employees using Native Query

Aim

To retrieve employee records using Native SQL Query in Spring Data JPA.

Objectives

1. To understand Native Queries.
 2. To execute SQL directly from Spring Data JPA.
 3. To use `nativeQuery=true`.
 4. To compare HQL and Native Query.
 5. To retrieve employee details using SQL.
-

Prerequisites

- Java 17
 - Spring Boot
 - Spring Data JPA
 - Hibernate
 - MySQL
 - Maven
-

Theory

Native Queries allow developers to execute database-specific SQL statements directly.

Unlike HQL, Native Queries operate directly on database tables and columns.

Native queries are useful when:

- Complex SQL is required.
- Vendor-specific features are needed.
- Stored procedures are used.

However, excessive use of Native Queries should be avoided because they reduce database portability.

Architecture / Working Diagram

[Insert Native Query Architecture Diagram Here]

Scenario Description

An organization stores employee details in MySQL database. The application should retrieve all employee records using SQL query instead of HQL.

Project Structure

```
src
├── main
│   └── java
│       ├── entity
│       │   └── Employee.java
│       ├── repository
│       │   └── EmployeeRepository.java
│       ├── service
│       │   └── EmployeeService.java
│       └── OrmLearnApplication.java
```

Files Created

- Employee.java
 - EmployeeRepository.java
 - EmployeeService.java
 - OrmLearnApplication.java
-

Database Schema

Employee Table

Column

em_id

em_name

em_salary

em_permanent

em_dp_id

Implementation

Employee Entity

```
@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @Column(name = "em_id")
    private int id;

    @Column(name = "em_name")
    private String name;

    @Column(name = "em_salary")
    private double salary;

    private boolean permanent;
}
```

Repository Layer

```
@Repository
public interface EmployeeRepository
    extends JpaRepository<Employee,Integer> {

    @Query(value = "SELECT * FROM employee",
        nativeQuery = true)
    List<Employee> getAllEmployeesNative();
}
```

Service Layer

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repository;

    public List<Employee>
    getAllEmployeesNative() {

        return repository
            .getAllEmployeesNative();
    }
}
```

Controller/Test Layer

```
@PostConstruct
public void testNativeQuery() {

    List<Employee> employees =
        employeeService
            .getAllEmployeesNative();

    employees.forEach(System.out::println);
}
```

Main Application Class

```
@SpringBootApplication
public class OrmLearnApplication {

    public static void main(String[] args) {

        SpringApplication.run(
            OrmLearnApplication.class,
            args);
    }
}
```

Code Explanation

The repository method executes raw SQL directly on the database.

The `nativeQuery=true` attribute instructs Spring Data JPA to treat the query as SQL rather than HQL.

Native Query Explanation

```
SELECT * FROM employee
```

Explanation

Clause	Purpose
SELECT *	Selects all columns
employee	Database table name

Why nativeQuery=true is Used?

```
@Query(value="SELECT * FROM employee",  
        nativeQuery=true)
```

This attribute informs Spring Data JPA that the query is written in SQL.

Without this attribute, Hibernate treats the query as HQL.

Portability Concerns

Native Queries are database dependent.

A query written for MySQL may not work in Oracle or PostgreSQL without modifications.

Therefore, Native Queries should be used only when necessary.

HQL vs Native Query

Feature	HQL	Native Query
Uses Entities	Yes	No
Uses Tables	No	Yes
Portable	Yes	No
Vendor Specific	No	Yes

Execution Steps

1. Create Employee entity.
 2. Configure database.
 3. Create repository interface.
 4. Define native SQL query.
 5. Create service method.
 6. Execute application.
 7. Retrieve records.
 8. Verify output.
-

Sample Output

```
Employee{id=1,name='John'}  
  
Employee{id=2,name='Sam'}  
  
Employee{id=3,name='Peter'}
```

Screenshots

[Insert Screenshot 1 Here]

[Insert Screenshot 2 Here]

[Insert Screenshot 3 Here]

Advantages

1. Supports complex SQL.
 2. Uses vendor-specific features.
 3. High flexibility.
 4. Supports stored procedures.
-

Limitations

1. Database dependent.
 2. Reduced portability.
 3. Difficult maintenance.
-

Viva Questions and Answers

Q1. What is Native Query?

Native Query is direct SQL executed against the database.

Q2. Which attribute enables native query?

`nativeQuery=true`

Q3. Is Native Query portable?

No.

Q4. Which annotation defines native query?

`@Query`

Q5. When should Native Queries be used?

When complex SQL or vendor-specific features are required.

Result

Employee records were successfully retrieved from the database using Native SQL Query in Spring Data JPA.

EXPERIMENT NO. 6

TITLE: Criteria Query

Aim

To implement dynamic filtering using Criteria Query in Spring Data JPA.

Objectives

1. To understand the need for Criteria Query.
 2. To implement dynamic search conditions.
 3. To use `CriteriaBuilder` for query creation.
 4. To use `CriteriaQuery` and `Root` classes.
 5. To construct dynamic predicates programmatically.
-

Prerequisites

- Java 17
 - Spring Boot 3.x
 - Spring Data JPA
 - Hibernate ORM
 - MySQL
 - Maven
 - IntelliJ IDEA/Eclipse
-

Theory

In real-world applications, users often search data using multiple optional filters.

Consider an e-commerce website such as Amazon where users search for laptops and apply filters like:

- RAM Size
- Operating System
- Processor Type
- Weight
- Hard Disk Size
- Customer Rating

Since users may choose any combination of filters, writing fixed HQL queries becomes difficult.

Criteria API solves this problem by allowing queries to be constructed dynamically during runtime.

The Criteria API is type-safe and object-oriented.

The main components of Criteria API are:

1. CriteriaBuilder

Used to create various query objects and conditions.

2. CriteriaQuery

Represents the query definition.

3. Root

Represents the entity involved in the query.

4. Predicate

Represents filtering conditions.

Advantages of Criteria Query:

- Dynamic query generation.
- Type safety.
- Easy maintenance.
- Eliminates string-based query errors.

Architecture / Working Diagram

[Insert Criteria API Working Diagram Here]

Scenario Description

An online shopping portal allows users to search products by applying multiple filters.

Users may search products using:

- RAM Size
- Operating System
- Processor
- Price Range

Since all filters are optional, dynamic query generation is required.

Project Structure



Files Created

- Product.java
- ProductRepository.java
- ProductService.java
- OrmLearnApplication.java
- application.properties

Database Schema

PRODUCT TABLE

Column Name
product_id
product_name
ram
os
processor

Column Name

price

Entity Relationship Explanation

Only a single Product entity is used in this experiment.

Each product contains multiple searchable attributes.

Implementation

Product Entity

```
@Entity
@Table(name = "product")
public class Product {

    @Id
    @Column(name = "product_id")
    private int id;

    @Column(name = "product_name")
    private String name;

    private String ram;

    private String os;

    private String processor;

    private double price;

    public Product() {}

    public Product(int id,
                   String name,
                   String ram,
                   String os,
                   String processor,
                   double price) {

        this.id = id;
        this.name = name;
        this.ram = ram;
        this.os = os;
        this.processor = processor;
        this.price = price;
    }
}
```

```
// Getters and Setters  
}
```

Repository Layer

```
@Repository  
public interface ProductRepository  
    extends JpaRepository<Product,Integer> {  
  
}
```

Service Layer

```
@Service  
public class ProductService {  
  
    @PersistenceContext  
    private EntityManager entityManager;  
  
    public List<Product> searchProducts(  
        String ram,  
        String os,  
        String processor) {  
  
        CriteriaBuilder cb =  
            entityManager.getCriteriaBuilder();  
  
        CriteriaQuery<Product> cq =  
            cb.createQuery(Product.class);  
  
        Root<Product> root =  
            cq.from(Product.class);  
  
        List<Predicate> predicates =  
            new ArrayList<>();  
  
        if (ram != null && !ram.isEmpty()) {  
            predicates.add(  
                cb.equal(root.get("ram"), ram));  
        }  
  
        if (os != null && !os.isEmpty()) {  
            predicates.add(  
                cb.equal(root.get("os"), os));  
        }  
    }  
}
```

```
        if (processor != null &&
            !processor.isEmpty()) {

            predicates.add(
                cb.equal(
                    root.get("processor"),
                    processor));
        }

        cq.where(predicates.toArray(
            new Predicate[0]));

        TypedQuery<Product> query =
            entityManager.createQuery(cq);

        return query.getResultList();
    }
}
```

Controller/Test Layer

```
@PostConstruct
public void testCriteriaQuery() {

    List<Product> products =
        productService.searchProducts(
            "16GB",
            "Windows",
            "Intel i7");

    products.forEach(System.out::println);
}
```

Main Application Class

```
@SpringBootApplication
public class OrmLearnApplication {

    public static void main(String[] args) {

        SpringApplication.run(
            OrmLearnApplication.class,
            args);
    }
}
```

application.properties

```
spring.datasource.url=
jdbc:mysql://localhost:3306/ormlearn

spring.datasource.username=root
spring.datasource.password=root

spring.jpa.show-sql=true

spring.jpa.hibernate.ddl-auto=update

spring.jpa.properties.hibernate.format_sql=true
```

pom.xml Dependencies

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-data-jpa
  </artifactId>
</dependency>

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
</dependency>
```

Code Explanation

The Criteria API dynamically creates SQL statements at runtime.

Each filter selected by the user is converted into a Predicate.

Only selected conditions are added to the final query.

Criteria Query Explanation

Step 1: Create CriteriaBuilder

```
CriteriaBuilder cb =
    entityManager.getCriteriaBuilder();
```

The **CriteriaBuilder** object is used to create query components.

Step 2: Create CriteriaQuery

```
CriteriaQuery<Product> cq =  
    cb.createQuery(Product.class);
```

Defines the type of result expected from the query.

Step 3: Create Root

```
Root<Product> root =  
    cq.from(Product.class);
```

Represents the Product entity in the query.

Step 4: Create Predicates

```
predicates.add(  
    cb.equal(root.get("ram"), ram));
```

Predicates represent dynamic filtering conditions.

Step 5: Apply Predicates

```
cq.where(predicates.toArray(  
    new Predicate[0]));
```

Applies all selected filters to the query.

Step 6: Execute Query

```
TypedQuery<Product> query =  
    entityManager.createQuery(cq);
```

Executes the dynamically generated query.

Dynamic Filtering Example

User Selected Filters

```
RAM = 16GB  
OS = Windows  
Processor = Intel i7
```

Generated SQL

```
SELECT *  
FROM product  
WHERE ram='16GB'  
AND os='Windows'  
AND processor='Intel i7';
```

Execution Steps

1. Create Product entity.
2. Configure EntityManager.
3. Create CriteriaBuilder object.
4. Create CriteriaQuery object.
5. Create Root entity.
6. Add Predicates dynamically.
7. Execute TypedQuery.
8. Verify output.

Sample Input

```
RAM : 16GB  
  
OS : Windows  
  
Processor : Intel i7
```

Sample Output

```
Product{  
id=101,  
name='Dell Inspiron',  
ram='16GB',
```

```
os='Windows',  
processor='Intel i7',  
price=75000.0  
}  
  
Product{  
id=102,  
name='HP Pavilion',  
ram='16GB',  
os='Windows',  
processor='Intel i7',  
price=80000.0  
}
```

Screenshots

[Insert Screenshot 1 Here]

[Insert Screenshot 2 Here]

[Insert Screenshot 3 Here]

Advantages

1. Supports dynamic query generation.
 2. Type-safe API.
 3. Eliminates string-based errors.
 4. Easy maintenance and scalability.
-

Limitations

1. Syntax is verbose.
 2. Complex queries become lengthy.
 3. Difficult for beginners.
-

Viva Questions and Answers

Q1. What is Criteria Query?

Criteria Query is a type-safe API used for dynamic query generation.

Q2. What is CriteriaBuilder?

CriteriaBuilder is used to create query objects and conditions.

Q3. What is Root in Criteria API?

Root represents the entity participating in the query.

Q4. What is Predicate?

Predicate represents filtering conditions.

Q5. Why is Criteria API preferred?

Because it supports dynamic and type-safe query generation.

Conclusion

Spring Data JPA provides powerful mechanisms for database interaction using HQL, JPQL, Native Queries, and Criteria API.

Fetch joins improve performance by reducing unnecessary SQL queries, while Criteria API enables dynamic and flexible query construction. These features make Spring Data JPA highly suitable for enterprise-level applications.

References

1. Hibernate Documentation: <https://docs.jboss.org/hibernate/orm/>
2. Spring Data JPA Documentation: <https://spring.io/projects/spring-data-jpa>
3. Oracle JPA Documentation: <https://docs.oracle.com/javaee/>
4. Baeldung Spring Data JPA Guide: <https://www.baeldung.com/spring-data-jpa>
5. Hibernate Criteria API Tutorial: <https://howtodoinjava.com/hibernate/>